Amane の Templates

目录

1 数据结构 3
1.1 并查集 3
1.2 树状数组 3
1.3 线段树 4
1.4 懒标记线段树 5
1.5 李超线段树 7
1.6 RMQ / Sparse Table 8
1.7 线性基 8
1.8 离散化 9
1.9 笛卡尔树 9
1.10 分块 10
1.11 莫队 10
1.12 珂朵莉树 11
1.13 组合哈希 12
2 数学 13
2.1 组合数（自动扩容） 13
2.2 快速幂 13
2.3 欧拉筛 13
2.4 埃式筛 14
2.5 线性求逆元 14
2.6 扩展欧几里得 14
2.7 中国剩余定理 14
2.8 卢卡斯定理 14
2.9 高斯消元 15
2.10 矩阵快速幂 15
2.11 曼哈顿距离与切比雪夫距离 15
2.12 数论分块 15
2.13 计算几何 16
2.14 素数测试与因式分解 18
2.15 多项式 / NTT 19
2.16 快速 GCD 22
2.17 向下取整与向上取整 23
3 字符串 24
3.1 KMP 24
3.2 字符串哈希（随机底数） 24
3.3 字符串哈希（随机底数与模数） 25
3.4 字典树 25
3.5 01 字典树 25
3.6 Z 函数 26
3.7 Manacher 26
4 图与树 27
4.1 Dijkstra 27
4.2 Floyd 27
4.3 SPFA 27
4.4 差分约束 28
4.5 LCA（倍增） 28
4.6 LCA（DFS 序） 28
4.7 树链剖分 29
4.8 克鲁斯卡尔重构树 30
4.9 树哈希 30
4.10 树的重心 31
4.11 Hierholzer（无向图） 31
4.12 Hierholzer（有向图） 32
4.13 强连通分量 33
5 常用工具 34
5.1 chmin / chmax 34
5.2 int128 输入输出与运算 34
5.3 编译脚本 34
5.4 对拍（Linux） 35
5.5 随机数据生成 35

1 数据结构

DATA STRUCTURES

1.1 并查集

```cpp
struct DSU {
    vector<int> p, siz;
    DSU() {}
    DSU(int n) {
        init(n);
    }
    void init(int n) {
        p.resize(n + 1);
        iota(p.begin(), p.end(), 0);
        siz.assign(n + 1, 1);
    }
    int find(int x) {
        if (x != p[x]) {
            p[x] = find(p[x]);
        }
        return p[x];
    }
    bool merge(int x, int y) {
        x = find(x);
        y = find(y);
        if (x == y) return false;
        siz[x] += siz[y];
        p[y] = x;
        return true;
    }
    int size(int x) {
        return siz[find(x)];
    }
    bool same(int x, int y) {
        return find(x) == find(y);
    }
};
```

1.2 树状数组

```cpp
template <class T>
struct Fenwick {
    int n;
    vector<T> a;
    Fenwick() {}
    Fenwick(int N): n(N), a(N + 1) {}
    void add(int x, const T &v) {
        for (int i = x; i <= n; i += (i & -i)) {
            a[i] = a[i] + v;
        }
    }
    void modify(int x, const T &v) {
        add(x, v - rangeQuery(x, x));
    }
    T query(int x) {
        T ans {};
        for (int i = x; i > 0; i -= (i & -i)) {
            ans = ans + a[i];
        }
        return ans;
    }
    T rangeQuery(int l, int r) {
        return query(r) - query(l - 1);
    }
    int lower_bound(T v) {
        int x = 0;
        for (int i = 1 << __lg(n); i > 0; i >>= 1) {
            if (x + i <= n && a[x + i] < v) {
                x += i;
                v = v - a[x];
            }
        }
        return x + 1;
    }
    int upper_bound(T v) {
        int x = 0;
        for (int i = 1 << __lg(n); i > 0; i >>= 1) {
            if (x + i <= n && a[x + i] <= v) {
                x += i;
                v = v - a[x];
            }
        }
```

```
        return x + 1;
    }
};
```

1.3 线段树

```
template <class Info>
struct SegmentTree {
    int n;
    vector<Info> info;
    SegmentTree(): n(0) {}
    SegmentTree(int N, Info v = Info()) {
        init(vector<Info>(N + 1, v));
    }
    SegmentTree(const vector<Info> &a) {
        init(a);
    }
    void init(const vector<Info> &a) {
        n = int(a.size()) - 1;
        info.assign(n << 2, Info());
        auto build = [&](auto &&self, int id, int l, int r) {
            if (l == r) {
                info[id] = a[l];
                return;
            }
            int mid = (l + r) >> 1;
            self(self, id * 2, l, mid);
            self(self, id * 2 + 1, mid + 1, r);
            pushUp(id);
        };
        build(build, 1, 1, n);
    }
    void pushUp(int id) {
        info[id] = info[id * 2] + info[id * 2 + 1];
    }
    void modify(int id, int l, int r, int x, const Info &v) {
        if (l == r) {
            info[id] = v;
            return;
        }
        int mid = (l + r) >> 1;
        if (x <= mid){
            modify(id * 2, l, mid, x, v);
        } else {
            modify(id * 2 + 1, mid + 1, r, x, v);
        }
        pushUp(id);
    }
    void modify(int x, const Info &v) {
        modify(1, 1, n, x, v);
    }
    Info rangeQuery(int id, int l, int r, int x, int y) {
        if (x > r || y < l) {
            return Info();
        }
        if (x <= l && y >= r) {
            return info[id];
        }
        int mid = (l + r) >> 1;
        return rangeQuery(id * 2, l, mid, x, y) + rangeQuery(id * 2 + 1, mid + 1, r, x, y);
    }
    Info rangeQuery(int l, int r) {
        return rangeQuery(1, 1, n, l, r);
    }
    template<class F>
    int findFirst(int id, int l, int r, int x, int y, F &&pred) {
        if (x > r || y < l) {
            return -1;
        }
        if (x <= l && y >= r && !pred(info[id])) {
            return -1;
        }
        if (l == r) {
            return l;
        }
        int mid = (l + r) >> 1;
        int res = findFirst(id * 2, l, mid, x, y, pred);
        if (res == -1) {
            res = findFirst(id * 2 + 1, mid + 1, r, x, y, pred);
        }
        return res;
    }
    template<class F>
    int findFirst(int l, int r, F &&pred) {
```

```
        return findFirst(1, 1, n, l, r, pred);
    }
    template<class F>
    int findLast(int id, int l, int r, int x, int y, F &&pred) {
        if (x > r || y < l) {
            return -1;
        }
        if (x <= l && y >= r && !pred(info[id])) {
            return -1;
        }
        if (l == r) {
            return l;
        }
        int mid = (l + r) >> 1;
        int res = findLast(id * 2 + 1, mid + 1, r, x, y, pred);
        if (res == -1) {
            res = findLast(id * 2, l, mid, x, y, pred);
        }
        return res;
    }
    template<class F>
    int findLast(int l, int r, F &&pred) {
        return findLast(1, 1, n, l, r, pred);
    }
};

struct Info {
    bool status = false;

    friend Info operator+(const Info &a, const Info &b) {
        if (!a.status) {
            return b;
        }
        if (!b.status) {
            return a;
        }
        Info c;
        c.status = true;

        return c;
    }

};
```

1.4 懒标记线段树

> 维护幺半群信息时需正确设置幺元；`Info::apply` 与 `Tag::apply` 应保持标记语义一致。

```
template <class Info, class Tag>
struct LazySegmentTree {
    int n;
    vector<Info> info;
    vector<Tag> tag;
    LazySegmentTree(): n(0) {}
    LazySegmentTree(int N, Info v = Info()) {
        init(vector<Info>(N + 1, v));
    }
    LazySegmentTree(const vector<Info> &a) {
        init(a);
    }
    void init(const vector<Info> &a) {
        n = int(a.size()) - 1;
        info.assign(n << 2, Info());
        tag.assign(n << 2, Tag());
        auto build = [&](auto &&self, int id, int l, int r) {
            if (l == r) {
                info[id] = a[l];
                return;
            }
            int mid = (l + r) >> 1;
            self(self, id * 2, l, mid);
            self(self, id * 2 + 1, mid + 1, r);
            pushUp(id);
        };
        build(build, 1, 1, n);
    }
    void apply(int id, const Tag &t) {
        info[id].apply(t);
        tag[id].apply(t);
    }
    void pushDown(int id) {
        apply(id * 2, tag[id]);
        apply(id * 2 + 1, tag[id]);
```

```
        tag[id] = Tag();
    }
    void pushUp(int id) {
        info[id] = info[id * 2] + info[id * 2 + 1];
    }
    void modify(int id, int l, int r, int x, const Info &v) {
        if (l == r) {
            info[id] = v;
            return;
        }
        int mid = (l + r) >> 1;
        pushDown(id);
        if (x <= mid){
            modify(id * 2, l, mid, x, v);
        } else {
            modify(id * 2 + 1, mid + 1, r, x, v);
        }
        pushUp(id);
    }
    void modify(int x, const Info &v) {
        modify(1, 1, n, x, v);
    }
    Info rangeQuery(int id, int l, int r, int x, int y) {
        if (x > r || y < l) {
            return Info();
        }
        if (x <= l && y >= r) {
            return info[id];
        }
        pushDown(id);
        int mid = (l + r) >> 1;
        return rangeQuery(id * 2, l, mid, x, y) + rangeQuery(id * 2 + 1, mid + 1, r, x, y);
    }
    Info rangeQuery(int l, int r) {
        return rangeQuery(1, 1, n, l, r);
    }
    void rangeApply(int id, int l, int r, int x, int y, const Tag &t) {
        if (x > r || y < l) {
            return;
        }
        if (x <= l && y >= r) {
            apply(id, t);
            return;
        }
        pushDown(id);
        int mid = (l + r) >> 1;
        rangeApply(id * 2, l, mid, x, y, t);
        rangeApply(id * 2 + 1, mid + 1, r, x, y, t);
        pushUp(id);
    }
    void rangeApply(int l, int r, const Tag &t) {
        rangeApply(1, 1, n, l, r, t);
    }
    template<class F>
    int findFirst(int id, int l, int r, int x, int y, F &&pred) {
        if (x > r || y < l) {
            return -1;
        }
        if (x <= l && y >= r && !pred(info[id])) {
            return -1;
        }
        if (l == r) {
            return l;
        }
        pushDown(id);
        int mid = (l + r) >> 1;
        int res = findFirst(id * 2, l, mid, x, y, pred);
        if (res == -1) {
            res = findFirst(id * 2 + 1, mid + 1, r, x, y, pred);
        }
        return res;
    }
    template<class F>
    int findFirst(int l, int r, F &&pred) {
        return findFirst(1, 1, n, l, r, pred);
    }
    template<class F>
    int findLast(int id, int l, int r, int x, int y, F &&pred) {
        if (x > r || y < l) {
            return -1;
        }
        if (x <= l && y >= r && !pred(info[id])) {
            return -1;
        }
```

```
        if (l == r) {
            return l;
        }
        pushDown(id);
        int mid = (l + r) >> 1;
        int res = findLast(id * 2 + 1, mid + 1, r, x, y, pred);
        if (res == -1) {
            res = findLast(id * 2, l, mid, x, y, pred);
        }
        return res;
    }
    template<class F>
    int findLast(int l, int r, F &&pred) {
        return findLast(1, 1, n, l, r, pred);
    }
};

struct Tag {
    bool status = false;

    void apply(const Tag &t) {
        if (!t.status) {
            return;
        }
        if (!status) {
            *this = t;
            return;
        }

    }
};

struct Info {
    bool status = false;

    void apply(const Tag &t) {
        if (!t.status) {
            return;
        }

    }

    friend Info operator+(const Info &a, const Info &b) {
        if (!a.status) {
            return b;
        }
        if (!b.status) {
            return a;
        }
        Info c;
        c.status = true;

        return c;
    }

};
```

1.5 李超线段树

```
constexpr i64 inf = 2e18;
template <class T>
struct LiChaoTree {
  struct Line {
    T a, b;
    Line(): a(0), b(-inf) {
    }
    Line(T a, T b): a(a), b(b) {
    }
    T get(T x) {
      return a * x + b;
    }
  };
  int N;
  vector<T> x;
  vector<Line> ST;
  LiChaoTree() {}
  LiChaoTree(const vector<T> &x2) {
    x = x2;
    sort(x.begin(), x.end());
    x.erase(unique(x.begin(), x.end()), x.end());
    int N2 = x.size();
    N = 1;
    while (N < N2) {
      N *= 2;
```

```
    }
    x.resize(N);
    for (int i = N2; i < N; i++) {
      x[i] = x[N2 - 1];
    }
    ST = vector<Line>(N * 2 - 1);
  }
  void addLine(Line L, int i, int l, int r) {
    T la = L.get(x[l]);
    T lb = ST[i].get(x[l]);
    T ra = L.get(x[r - 1]);
    T rb = ST[i].get(x[r - 1]);
    if (la <= lb && ra <= rb) {
      return;
    } else if (la >= lb && ra >= rb) {
      ST[i] = L;
    } else {
      int m = (l + r) / 2;
      T ma = L.get(x[m]);
      T mb = ST[i].get(x[m]);
      if (ma > mb) {
        swap(L, ST[i]);
        swap(la, lb);
        swap(ra, rb);
      }
      if (la > lb) {
        addLine(L, i * 2 + 1, l, m);
      }
      if (ra > rb) {
        addLine(L, i * 2 + 2, m, r);
      }
    }
  }
  void addLine(T a, T b) {
    addLine(Line(a, b), 0, 0, N);
  }
  T getMax(T x2) {
    int p = lower_bound(x.begin(), x.end(), x2) - x.begin();
    p += N - 1;
    T ans = -inf;
    ans = max(ans, ST[p].get(x2));
    while (p > 0) {
      p = (p - 1) / 2;
      ans = max(ans, ST[p].get(x2));
    }
    return ans;
  }
};
```

1.6 RMQ / Sparse Table

```
template<class T, class F>
struct RMQ {
    int n;
    vector<T> a;
    array<vector<T>, 20> f;
    F fun;
    RMQ() {}
    RMQ(const vector<T> &a_, F &&fun_): a(a_), fun(fun_) {
        n = int(a.size()) - 1;
        f.fill(vector<T>(n + 1));
        for (int i = 1; i <= n; i++) {
            f[0][i] = a[i];
        }
        for (int j = 1; j <= __lg(n); j++) {
            for (int i = 1; i + (1 << j) - 1 <= n; i++) {
                f[j][i] = fun(f[j - 1][i], f[j - 1][i + (1 << (j - 1))]);
            }
        }
    }
    T query(int l, int r) {
        int k = __lg(r - l + 1);
        return fun(f[k][l], f[k][r - (1 << k) + 1]);
    }
};
```

1.7 线性基

```
template <class T>
struct LinearBasis {
    static constexpr int N = __lg(numeric_limits<T>::max());
    array<T, N + 1> b;
    bool zero;
```

```
    LinearBasis() {
        zero = false;
        b.fill(0);
    }

    void insert(T x) {
        for (int i = N; i >= 0; i--) {
            if (x >> i & 1) {
                if (b[i] == 0) {
                    b[i] = x;
                    return;
                }
                x ^= b[i];
            }
        }
        zero = true;
    }

    T queryMax() {
        T ans = 0;
        for (int i = N; i >= 0; i--) {
            ans = max(ans, ans ^ b[i]);
        }
        return ans;
    }

    T queryMin() {
        if (zero) {
            return T(0);
        }
        T res;
        for (int i = 0; i <= N; i++) {
            if (b[i] != 0) {
                res = b[i];
                break;
            }
        }
        return res;
    }

    bool check(T x) {
        for (int i = N; i >= 0; i--) {
            if (x >> i & 1) {
                if (b[i] == 0) {
                    return false;
                }
                x ^= b[i];
            }
        }
        return true;
    }
};
```

1.8 离散化

> `offset` 决定参与排序去重的起始下标；1 索引数组应传入 1。

```
template <class T>
vector<T> sparse(const vector<T> &a, int offset) {
    auto sp = a;
    sort(sp.begin() + offset, sp.end());
    sp.erase(unique(sp.begin() + offset, sp.end()), sp.end());
    return sp;
}
```

1.9 笛卡尔树

```
vector<int> stk;
vector<int> lc(n + 1), rc(n + 1);
for (int i = 1; i <= n; i++) {
    while (!stk.empty() && a[i] < a[stk.back()]) {
        lc[i] = stk.back();
        stk.pop_back();
    }
    if (!stk.empty()) {
        rc[stk.back()] = i;
    }
    stk.push_back(i);
}
```

1.10 分块

示例维护区间加与区间和。

```
int n, q;
    cin >> n >> q;
    vector<ll> a(n + 1);
    for (int i = 1; i ≤ n; i++) {
        cin >> a[i];
    }
    const int B = sqrt(n);
    vector<int> id(n + 1);
    // m代表最后一个块的编号
    int m = (n + B - 1) / B;
    // 维护每个块内的区间和
    vector<ll> s(m + 1);
    // 维护每个块的区间加
    vector<ll> add(m + 1);
    for (int i = 1; i ≤ n; i++) {
        id[i] = (i + B - 1) / B;
        s[id[i]] += a[i];
    }
    while (q--) {
        int op;
        cin >> op;
        if (op == 1) {
            int x, y;
            ll k;
            cin >> x >> y >> k;
            // 如果区间在同一块内，暴力修改即可
            if (id[x] == id[y]) {
                for (int i = x; i ≤ y; i++) {
                    a[i] += k;
                    s[id[i]] += k;
                }
            } else { // 不在同一块内，先将左右端点所在块内的元素暴力修改
                for (int i = x; id[i] == id[x]; i++) {
                    a[i] += k;
                    s[id[i]] += k;
                }
                for (int i = y; id[i] == id[y]; i--) {
                    a[i] += k;
                    s[id[i]] += k;
                }
                // 剩下的都是完成块，暴力遍历每一块即可
                for (int i = id[x] + 1; i < id[y]; i++) {
                    s[i] += k * B;
                    add[i] += k;
                }
            }
        } else {
            int x, y;
            cin >> x >> y;
            ll res = 0;
            if (id[x] == id[y]) {
                for (int i = x; i ≤ y; i++) {
                    res += a[i] + add[id[i]];
                }
            } else {
                for (int i = x; id[i] == id[x]; i++) {
                    res += a[i] + add[id[i]];
                }
                for (int i = y; id[i] == id[y]; i--) {
                    res += a[i] + add[id[i]];
                }
                for (int i = id[x] + 1; i < id[y]; i++) {
                    res += s[i];
                }
            }
            cout << res << '\n';
        }
    }
    return 0;
```

1.11 莫队

示例为离线查询区间不同数字数量。

```
int n, q;
cin >> n >> q;
vector<int> a(n + 1);
for (int i = 1; i ≤ n; i++) {
    cin >> a[i];
```

```
}

// 分块，记录每个索引属于的块的编号
const int B = sqrt(n);
vector<int> bel(n + 1);
for (int i = 1; i <= n; i++) {
    bel[i] = (i + B - 1) / B;
}

// 将询问离线
vector<array<int, 3>> Q(q);
for (int i = 0; i < q; i++) {
    int l, r;
    cin >> l >> r;
    Q[i] = {l, r, i};
}

// 按左端点所在块的编号为第一关键字，右端点为第二关键字排序
sort(all(Q), [&](const auto &a, const auto &b) {
    if (bel[a[0]] != bel[b[0]]) {
        return bel[a[0]] < bel[b[0]];
    }
    // 奇偶优化，减少时间常数
    if (bel[a[0]] & 1) {
        return a[1] < b[1];
    } else {
        return a[1] > b[1];
    }
});

// 记录区间内不同数字出现的次数
vector<int> cnt(n + 1);
int cur = 0;

auto add = [&](int k) -> void {
    if (cnt[a[k]]++ == 0) {
        cur++;
    }
};

auto del = [&](int k) -> void {
    if (--cnt[a[k]] == 0) {
        cur--;
    }
};

// 记录每个询问的答案
vector<int> ans(q);

for (int i = 0, l = 1, r = 0; i < q; i++) {
    auto [x, y, id] = Q[i];
    while (l > x) add(--l); // 左扩展
    while (r < y) add(++r); // 右扩展
    while (l < x) del(l++); // 左删除
    while (r > y) del(r--); // 右删除
    ans[id] = cur;
}

for (auto x : ans) {
    cout << x << '\n';
}
```

1.12 珂朵莉树

```
struct Node {
    int l, r;
    mutable int v;
    Node(int L, int R = 0, int V = 0): l(L), r(R), v(V) {}
    bool operator<(const Node &o) const {
        return l < o.l;
    }
};

set<Node> odt;
odt.insert(Node(1, 1e5 + 1, 0));
auto split = [&](int pos) -> IT {
    auto it = odt.lower_bound(Node(pos));
    if (it != odt.end() && it->l == pos) {
        return it;
    }
    it--;
    int L = it->l, R = it->r, V = it->v;
    odt.erase(it);
    odt.insert(Node(L, pos - 1, V));
```

```
    return odt.insert(Node(pos, R, V)).first;
};

auto assign = [&](int l, int r, int v) -> void {
    auto itr = split(r + 1), itl = split(l);
    for (auto it = itl; it =/= itr; it++) {
        // ..
    }
    M[v] += r - l + 1;
    odt.erase(itl, itr);
    odt.insert(Node(l, r, v));
};
```

1.13 组合哈希

```
template <class T>
void hash_combine(std::size_t& seed, const T& v) {
    seed ^= std::hash<T>{}(v)
          + 0x9e3779b97f4a7c15ULL
          + (seed << 6)
          + (seed >> 2);
}

struct Hash {
    size_t operator()(const array<int, 2> &a) const {
        size_t res = 0;
        for (auto &e : a) {
            hash_combine(res, e);
        }
        return res;
    };
};

unordered_map<array<int, 2>, int, Hash> M;
```

2 数学

MATHEMATICS

2.1 组合数（自动扩容）

```cpp
constexpr int mod = 998244353;
i64 power(i64 a, i64 b) {
    i64 res = 1;
    while (b) {
        if (b & 1) res = res * a % mod;
        b >>= 1;
        a = a * a % mod;
    }
    return res;
}

struct Comb {
    int n;
    vector<i64> _fac, _infac, _inv;
    Comb(): n{0}, _fac{1}, _infac{1}, _inv{0} {}
    Comb(int m): Comb() {
        init(m);
    }
    void init(int m) {
        if (m <= n) return;
        _fac.resize(m + 1);
        _infac.resize(m + 1);
        _inv.resize(m + 1);
        for (int i = n + 1; i <= m; i++) {
            _fac[i] = _fac[i - 1] * i % mod;
        }
        _infac[m] = power(_fac[m], mod - 2);
        for (int i = m; i > n; i--) {
            _infac[i - 1] = _infac[i] * i % mod;
            _inv[i] = _infac[i] * _fac[i - 1] % mod;
        }
        n = m;
    }
    i64 fac(int m) {
        if (m > n) init(m * 2);
        return _fac[m];
    }
    i64 infac(int m) {
        if (m > n) init(m * 2);
        return _infac[m];
    }
    i64 inv(int m) {
        if (m > n) init(m * 2);
        return _inv[m];
    }
    i64 binom(int a, int b) {
        if (a < b || b < 0) return 0ll;
        return fac(a) * infac(a - b) % mod * infac(b) % mod;
    }
} comb;
```

2.2 快速幂

```cpp
constexpr int mod = 998244353;
i64 power(i64 a, i64 b) {
    i64 res = 1;
    while (b) {
        if (b & 1) {
            res = res * a % mod;
        }
        a = a * a % mod;
        b >>= 1;
    }
    return res;
}
```

2.3 欧拉筛

```cpp
vector<int> prime, minp;
void sieve(int n) {
    minp.assign(n + 1, 0);
    prime.clear();

    for (int i = 2; i <= n; i++) {
        if (minp[i] == 0) {
            minp[i] = i;
```

```
            prime.push_back(i);
        }

        for (auto p : prime) {
            if (1LL * p * i > n) {
                break;
            }
            minp[p * i] = p;
            if (minp[i] == p) {
                break;
            }
        }
    }
}
```

2.4 埃式筛

```
constexpr int N = 1e8;
bitset<N + 1> is;
vector<int> prime;

void ErSieve() {
    for (int i = 2; i <= N; i++) {
        if (is[i]) continue;
        prime.push_back(i);
        for (ll j = 1LL * i * i; j <= N; j += i) {
            is.set(j);
        }
    }
}
```

2.5 线性求逆元

> 模数 P 必须为质数。

```
int n, P;
cin >> n >> P;
vector<ll> inv(n + 1);
inv[1] = 1;
for (int i = 2; i <= n; i++) {
    inv[i] = (P - P / i) * inv[P % i] % P;
}
```

2.6 扩展欧几里得

```
template <class T>
array<T, 3> exgcd(const T &a, const T &b) {
    if (b == T(0)) {
        return {a, T(1), T(0)};
    }
    auto [g, x, y] = exgcd(b, a % b);
    return {g, y, x - a / b * y};
}
```

2.7 中国剩余定理

```
template<class T>
T CRT(const std::vector<T> &m, const std::vector<T> &r) {
    T M = 1, ans = 0;
    for (int i = 0; i < m.size(); i++)M *= m[i];
    for (int i = 0; i < m.size(); i++) {
        T c = M / m[i];
        __int128 x, y;
        exgcd(c, m[i], x, y);
        ans = (ans + r[i] * c * x % M) % M;
    }
    return (ans % M + M) % M;
}
```

2.8 卢卡斯定理

```
i64 Lucas(i64 n, i64 m) {
    if (m < 0 || m > n) return 0;
    if (m == 0) return 1;

    i64 ni = n % mod;
    i64 mi = m % mod;
    if (mi > ni) return 0;
```

```
    return (i128)comb.binom(ni, mi) * Lucas(n / mod, m / mod, mod) % mod;
}
```

2.9 高斯消元

```
bool gauss(vector<vector<int>> &a) {
    int n = (int)a.size() - 1;
    for (int i = 1; i <= n; i++) {
        int r = i;
        for (int k = i; k <= n; k++) {
            if (fabs(a[k][i]) > eps) {
                r = k;
                break;
            }
        }
        if (r != i) swap(a[i], a[r]);
        if (fabs(a[i][i]) < eps) return false;

        for (int j = n + 1; j >= i; j--) {
            a[i][j] /= a[i][i];
        }
        for (int k = i + 1; k <= n; k++) {
            for (int j = n + 1; j >= i; j--) {
                a[k][j] -= a[i][j] * a[k][i];
            }
        }
    }
    for (int i = n - 1; i >= 1; i--) {
        for (int j = i + 1; j <= n; j++) {
            a[i][n + 1] -= a[i][j] * a[j][n + 1];
        }
    }
    return true;
}
```

2.10 矩阵快速幂

```
constexpr int N = 2;
constexpr int mod = 1e9 + 7;
using Mat = array<array<i64, N>, N>;

Mat operator*(const Mat &a, const Mat &b) {
    Mat res {};
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            for (int k = 0; k < N; k++) {
                res[i][j] += a[i][k] * b[k][j] % mod;
                res[i][j] %= mod;
            }
        }
    }
    return res;
}

template <class T>
T power(T a, i64 b) {
    Mat res {};
    for (int i = 0; i < N; i++) {
        res[i][i] = 1;
  }

    while (b) {
        if (b & 1) {
            res = res * a;
        }
        b >>= 1;
        a = a * a;
    }
    return res;
}
```

2.11 曼哈顿距离与切比雪夫距离

> 映射 (x, y) → (x + y, x - y) 后，原坐标系的曼哈顿距离变为新坐标系的切比雪夫距离。 反向映射可写为 ((x + y) / 2, (x - y) / 2)。

2.12 数论分块

```
ll division_block(ll n) {
    ll res = 0;
    for (ll l = 1, r; l <= n; l = r + 1) {
```

```
        r = n / (n / l);
        res += n / l * (r - l + 1);
    }
    return res;
}
```

2.13 计算几何

```
using F = long double;
constexpr F eps = 1e-8;

template <class T>
struct Point {
    T x, y;
    Point(const T &x_ = 0, const T &y_ = 0) : x(x_), y(y_) {}
    Point operator+(const Point &p) const {
        return {x + p.x, y + p.y};
    }
    Point operator-(const Point &p) const {
        return {x - p.x, y - p.y};
    }
    Point operator*(const T &v) const {
        return {x * v, y * v};
    }
    Point operator/(const T &v) const {
        return {x / v, y / v};
    }
};

template <class T>
struct Line {
    Point<T> a;
    Point<T> b;
    Line(const Point<T> &a_ = Point<T>(), const Point<T> &b_ = Point<T>()): a(a_), b(b_) {}
};

template <class T>
std::ostream &operator<<(std::ostream &os, const Point<T> &p) {
    return os << "(" << p.x << ", " << p.y << ")";
}

template <class T>
std::istream &operator>>(std::istream &is, Point<T> &p) {
    is >> p.x >> p.y;
    return is;
}

template <class T>
bool equal(const T &x, const T &y) {
    if constexpr (is_floating_point_v<T>) {
        return fabs(x - y) < eps;
    } else {
        return x == y;
    }
}

template <class T>
T dot(const Point<T> &a, const Point<T> &b) {
    return a.x * b.x + a.y * b.y;
}

template <class T>
T cross(const Point<T> &a, const Point<T> &b) {
    return a.x * b.y - a.y * b.x;
}

template <class T>
T square(const Point<T> &p) {
    return dot(p, p);
}

template <class T>
double length(const Point<T> &p) {
    return sqrt(square(p));
}

template <class T>
double length(const Line<T> &l) {
    return length(l.a - l.b);
}

template <class T>
bool parallel(const Line<T> &l1, const Line<T> &l2) {
    return equal(cross(l1.a - l1.b, l2.a - l2.b), T(0));
```

```
}

template <class T>
Point<T> normalize(const Point<T> &p) {
    return p / length(p);
}

template <class T>
double distance(const Point<T> &a, const Point<T> &b) {
    return length(b - a);
}

template <class T>
double distancePL(const Point<T> &p, const Line<T> &l) {
    return abs(cross(l.a - p, l.b - p)) / length(l);
}

template <class T>
double distancePS(const Point<T> &p, const Line<T> &l) {
    if (dot(p - l.a, l.b - l.a) < 0) {
        return distance(p, l.a);
    }
    if (dot(p - l.b, l.a - l.b) < 0) {
        return distance(p, l.b);
    }
    return distancePL(p, l);
}

template <class T>
Point<T> lineIntersection(const Line<T> &l1, const Line<T> &l2) {
    return l1.a + (l1.b - l1.a) * cross(l2.a - l1.a, l2.b - l2.a) / cross(l2.b - l2.a, l1.b - l1.a);
}

template <class T>
auto getHull(vector<Point<T>> ps) {
    sort(ps.begin(), ps.end(), [&](const auto &p1, const auto &p2) {
        return equal(p1.x, p2.x) ? p1.y < p2.y : p1.x < p2.x;
    });
    vector<Point<T>> hi, lo;
    for (auto &p : ps) {
        while (lo.size() > 1 && cross(lo.back() - lo[lo.size() - 2], p - lo.back()) <= 0) {
            lo.pop_back();
        }
        lo.push_back(p);

        while (hi.size() > 1 && cross(hi.back() - hi[hi.size() - 2], p - hi.back()) >= 0) {
            hi.pop_back();
        }
        hi.push_back(p);
    }

    return make_pair(lo, hi);
}

template<class T>
bool pointOnLineLeft(const Point<T> &p, const Line<T> &l) {
    return cross(l.b - l.a, p - l.a) > 0;
}

template<class T>
bool pointOnSegment(const Point<T> &p, const Line<T> &l) {
    return cross(p - l.a, l.b - l.a) == 0 && std::min(l.a.x, l.b.x) <= p.x && p.x <= std::max(l.a.x, l.b.x)
        && std::min(l.a.y, l.b.y) <= p.y && p.y <= std::max(l.a.y, l.b.y);
}

template<class T>
bool pointInPolygon(const Point<T> &a, const std::vector<Point<T>> &p) {
    int n = p.size();
    for (int i = 0; i < n; i++) {
        if (pointOnSegment(a, Line(p[i], p[(i + 1) % n]))) {
            return true;
        }
    }

    int t = 0;
    for (int i = 0; i < n; i++) {
        auto u = p[i];
        auto v = p[(i + 1) % n];
        if (u.x < a.x && v.x >= a.x && pointOnLineLeft(a, Line(v, u))) {
            t ^= 1;
        }
        if (u.x >= a.x && v.x < a.x && pointOnLineLeft(a, Line(u, v))) {
            t ^= 1;
        }
```

```
    }

    return t == 1;
}
```

2.14 素数测试与因式分解

```
using i64 = long long;
i64 mul(i64 a, i64 b, i64 m) {
    return static_cast<__int128>(a) * b % m;
}
i64 power(i64 a, i64 b, i64 m) {
    i64 res = 1 % m;
    for (; b; b >>= 1, a = mul(a, a, m))
        if (b & 1)
            res = mul(res, a, m);
    return res;
}
bool isprime(i64 n) {
    if (n < 2)
        return false;
    static constexpr int A[] = {2, 3, 5, 7, 11, 13, 17, 19, 23};
    int s = __builtin_ctzll(n - 1);
    i64 d = (n - 1) >> s;
    for (auto a : A) {
        if (a == n)
            return true;
        i64 x = power(a, d, n);
        if (x == 1 || x == n - 1)
            continue;
        bool ok = false;
        for (int i = 0; i < s - 1; ++i) {
            x = mul(x, x, n);
            if (x == n - 1) {
                ok = true;
                break;
            }
        }
        if (!ok)
            return false;
    }
    return true;
}
std::vector<i64> factorize(i64 n) {
    std::vector<i64> p;
    std::function<void(i64)> f = [&](i64 n) {
        if (n <= 10000) {
            for (int i = 2; i * i <= n; ++i)
                for (; n % i == 0; n /= i)
                    p.push_back(i);
            if (n > 1)
                p.push_back(n);
            return;
        }
        if (isprime(n)) {
            p.push_back(n);
            return;
        }
        auto g = [&](i64 x) {
            return (mul(x, x, n) + 1) % n;
        };
        i64 x0 = 2;
        while (true) {
            i64 x = x0;
            i64 y = x0;
            i64 d = 1;
            i64 power = 1, lam = 0;
            i64 v = 1;
            while (d == 1) {
                y = g(y);
                ++lam;
                v = mul(v, std::abs(x - y), n);
                if (lam % 127 == 0) {
                    d = std::gcd(v, n);
                    v = 1;
                }
                if (power == lam) {
                    x = y;
                    power *= 2;
                    lam = 0;
                    d = std::gcd(v, n);
                    v = 1;
                }
            }
```

```
            if (d != n) {
                f(d);
                f(n / d);
                return;
            }
            ++x0;
        }
    };
    f(n);
    std::sort(p.begin(), p.end());
    return p;
}
```

2.15 多项式 / NTT

```
constexpr i64 mod = 998244353;
constexpr i64 G = 3;

i64 power(i64 a, i64 b = mod - 2) {
    i64 res = 1;
    while (b) {
        if (b & 1) {
            res = res * a % mod;
        }
        a = a * a % mod;
        b >>= 1;
    }
    return res;
}

template<i64 mod, i64 G>
struct PolyNTT {
    // NTT 整数模意义下的多项式乘法
    static void ntt(vector<i64>& a, bool invert) {
        int n = a.size();
        vector<int> rev(n);
        for (int i = 0; i < n; i++) {
            rev[i] = (rev[i >> 1] >> 1) | ((i & 1) ? (n >> 1) : 0);
            if (i < rev[i]) swap(a[i], a[rev[i]]);
        }
        for (int len = 2; len <= n; len <<= 1) {
            i64 wlen = power(G, (mod - 1) / len);
            if (invert) wlen = power(wlen);
            for (int i = 0; i < n; i += len) {
                i64 w = 1;
                for (int j = 0; j < len / 2; j++) {
                    i64 u = a[i + j];
                    i64 v = a[i + j + len / 2] * w % mod;
                    a[i + j] = (u + v) % mod;
                    a[i + j + len / 2] = (u - v + mod) % mod;
                    w = w * wlen % mod;
                }
            }
        }
        if (invert) {
            i64 inv_n = power(n);
            for (i64 &x : a) x = x * inv_n % mod;
        }
    }

    // FFT 通用多项式乘法（实现为 NTT）
    static void fft(vector<i64>& a, bool invert) {
        ntt(a, invert);
    }

    // FWT 子集卷积 / 集合运算
    static void fwt(vector<i64>& a, bool invert) {
        int n = a.size();
        for (int len = 1; len < n; len <<= 1) {
            for (int i = 0; i < n; i++) {
                if (i & len) continue;
                i64 x = a[i], y = a[i + len];
                if (invert) {
                    // IFWT for XOR
                    a[i] = (x + y) % mod * power(2) % mod;
                    a[i + len] = (x - y + mod) % mod * power(2) % mod;
                } else {
                    // FWT for XOR
                    a[i] = (x + y) % mod;
                    a[i + len] = (x - y + mod) % mod;
                }
            }
        }
    }
```

```cpp
// 多项式加法
static vector<i64> add(vector<i64> a, vector<i64> b) {
    int n = max(a.size(), b.size());
    a.resize(n);
    b.resize(n);
    for (int i = 0; i < n; i++) {
        a[i] = (a[i] + b[i]) % mod;
    }
    return a;
}

// 多项式减法
static vector<i64> sub(vector<i64> a, vector<i64> b) {
    int n = max(a.size(), b.size());
    a.resize(n);
    b.resize(n);
    for (int i = 0; i < n; i++) {
        a[i] = (a[i] - b[i] + mod) % mod;
    }
    return a;
}

// 多项式乘法
static vector<i64> multiply(vector<i64> a, vector<i64> b) {
    if (a.empty() || b.empty()) return {};
    int res_deg = (int)a.size() + (int)b.size() - 2;
    int sz = 1;
    while (sz < (int)a.size() + (int)b.size()) sz <<= 1;
    a.resize(sz);
    b.resize(sz);
    ntt(a, false);
    ntt(b, false);
    for (int i = 0; i < sz; i++) a[i] = a[i] * b[i] % mod;
    ntt(a, true);
    a.resize(res_deg + 1);
    return a;
}

// 多项式除法（返回商）
static vector<i64> divide(const vector<i64>& a, const vector<i64>& b) {
    int n = a.size(), m = b.size();
    if (n < m) return {0};
    vector<i64> ra = a;
    vector<i64> rb = b;
    reverse(ra.begin(), ra.end());
    reverse(rb.begin(), rb.end());
    vector<i64> rb_inv;
    poly_inv(rb, rb_inv, n - m + 1);
    vector<i64> q = multiply(ra, rb_inv);
    q.resize(n - m + 1);
    reverse(q.begin(), q.end());
    return q;
}

// 多项式模除（返回余数）
static vector<i64> Mod(const vector<i64>& a, const vector<i64>& b) {
    if (a.size() < b.size()) return a;
    vector<i64> q = divide(a, b);
    vector<i64> r = sub(a, multiply(q, b));
    r.resize(min((int)r.size(), (int)b.size() - 1));
    return r;
}

// 多项式取逆，递归构造
static void poly_inv(const vector<i64>& a, vector<i64>& b, int deg) {
    if (deg == 1) {
        b.assign(1, power(a[0]));
        return;
    }
    poly_inv(a, b, (deg + 1) / 2);
    int sz = 1;
    while (sz < 2 * deg) sz <<= 1;
    vector<i64> a_slice(a.begin(), a.begin() + min((int)a.size(), deg));
    a_slice.resize(sz);
    b.resize(sz);
    ntt(a_slice, false);
    ntt(b, false);
    for (int i = 0; i < sz; i++) {
        b[i] = (2 * b[i] - a_slice[i] * b[i] % mod * b[i] % mod + mod) % mod;
    }
    ntt(b, true);
    b.resize(deg);
}
```

```cpp
    // 多项式求导
    static vector<i64> derivative(const vector<i64>& a) {
        int n = a.size();
        if (n <= 1) return {};
        vector<i64> res(n - 1);
        for (int i = 1; i < n; i++) res[i - 1] = a[i] * i % mod;
        return res;
    }

    // 多项式积分
    static vector<i64> integral(const vector<i64>& a) {
        int n = a.size();
        vector<i64> res(n + 1);
        vector<i64> inv(n + 2);
        inv[1] = 1;
        for (int i = 2; i <= n + 1; i++) inv[i] = mod - (mod / i) * inv[mod % i] % mod;
        for (int i = 0; i < n; i++) res[i + 1] = a[i] * inv[i + 1] % mod;
        return res;
    }

    // 多项式对数
    static void ln(const vector<i64>& a, vector<i64>& b, int deg) {
        vector<i64> a_der = derivative(a);
        vector<i64> a_inv;
        poly_inv(a, a_inv, deg);
        vector<i64> t = multiply(a_der, a_inv);
        t.resize(deg - 1);
        b = integral(t);
        b.resize(deg);
    }

    // 多项式指数
    static void exp(const vector<i64>& a, vector<i64>& b, int deg) {
        if (deg == 1) {
            b.assign(1, 1);
            return;
        }
        exp(a, b, (deg + 1) / 2);
        b.resize(deg);
        vector<i64> ln_b;
        ln(b, ln_b, deg);
        vector<i64> a_slice(a.begin(), a.begin() + min((int)a.size(), deg));
        a_slice.resize(deg, 0);
        for (int i = 0; i < deg; i++) {
            ln_b[i] = (a_slice[i] - ln_b[i] + mod) % mod;
        }
        ln_b[0] = (ln_b[0] + 1) % mod;
        b = multiply(b, ln_b);
        b.resize(deg);
    }

    // 多点求值
    static void build_eval_tree(vector<vector<i64>>& tree, const vector<i64>& xs, int u, int l, int r) {
        if (r - l == 1) {
            tree[u] = {(mod - xs[l]) % mod, 1};
        } else {
            int m = (l + r) / 2;
            build_eval_tree(tree, xs, 2 * u, l, m);
            build_eval_tree(tree, xs, 2 * u + 1, m, r);
            tree[u] = multiply(tree[2 * u], tree[2 * u + 1]);
        }
    }

    static void fast_eval_rec(const vector<i64>& f, const vector<vector<i64>>& tree, vector<i64>& res, int u, int l, int r) {
        if (f.size() < 256) { // 小范围暴力求值优化
            for (int i = l; i < r; ++i) {
                i64 x = res[i];
                i64 y = 0, p = 1;
                for (i64 coeff : f) {
                    y = (y + coeff * p) % mod;
                    p = p * x % mod;
                }
                res[i] = y;
            }
            return;
        }
        if (r - l == 1) {
            res[l] = f.size() ? f[0] : 0;
            return;
        }
        int m = (l + r) / 2;
        vector<i64> rem_l = Mod(f, tree[2 * u]);
        vector<i64> rem_r = Mod(f, tree[2 * u + 1]);
```

```
        fast_eval_rec(rem_l, tree, res, 2 * u, l, m);
        fast_eval_rec(rem_r, tree, res, 2 * u + 1, m, r);
    }

    static vector<i64> fast_eval(const vector<i64>& f, const vector<i64>& xs) {
        int n = xs.size();
        if (n == 0) return {};
        vector<vector<i64>> tree(4 * n);
        build_eval_tree(tree, xs, 1, 0, n);
        vector<i64> res = xs;
        fast_eval_rec(f, tree, res, 1, 0, n);
        return res;
    }

    // 快速插值
    static vector<i64> interpolate(const vector<i64>& xs, const vector<i64>& ys) {
        int n = xs.size();
        if (n == 0) return {};
        vector<vector<i64>> tree(4 * n);
        build_eval_tree(tree, xs, 1, 0, n);
        vector<i64> all_poly = tree[1];
        vector<i64> der = derivative(all_poly);
        vector<i64> val = fast_eval(der, xs);
        vector<i64> weights(n);
        for (int i = 0; i < n; i++) {
            weights[i] = ys[i] * power(val[i]) % mod;
        }

        function<vector<i64>(int, int, int)> solve = [&](int u, int l, int r) -> vector<i64> {
            if (r - l == 1) {
                return {weights[l]};
            }
            int m = (l + r) / 2;
            vector<i64> left = solve(2 * u, l, m);
            vector<i64> right = solve(2 * u + 1, m, r);
            return add(multiply(left, tree[2 * u + 1]), multiply(right, tree[2 * u]));
        };

        return solve(1, 0, n);
    }
};

using Poly = PolyNTT<mod, G>;
using poly = vector<i64>;
```

2.16 快速 GCD

```
struct FastGCD {
    int V;          // 值域上限
    int RADIO;      // 阈值，通常为 sqrt(V)

    vector<int> np;          // np[i] > 0 表示 i 是合数
    vector<int> prime;       // 存储找到的素数
    vector<array<int, 3>> k; // k[i] 存储 i 的一种特殊三因子分解
    vector<vector<int>> sg; // 预计算的小范围 GCD 表
    int cnt;                     // 找到的素数数量

    /**
     * @brief 构造函数，执行所有预处理操作。
     * @param n 预处理的最大值 V。
     *
     * 预处理时间复杂度近似为 O(V * log(logV)) + O(RADIO^2)。
     * 空间复杂度为 O(V)。
     */
    FastGCD(int n) : V(n), RADIO(static_cast<int>(floor(sqrt(n)))), cnt(0) {
        np.resize(n + 1);
        prime.resize(n + 1);
        k.resize(n + 1);
        sg.resize(RADIO + 1, vector<int>(RADIO + 1));

        k[1] = {1, 1, 1};
        np[1] = 1;

        for (int i = 2; i <= V; i++) {
            if (!np[i]) {
                prime[++cnt] = i;
                k[i] = {1, 1, i};
            }
            for (int j = 1; j <= cnt && 1LL * prime[j] * i <= V; j++) {
                np[i * prime[j]] = 1;
                auto &tmp = k[i * prime[j]];

                tmp[0] = k[i][0] * prime[j];
                tmp[1] = k[i][1];
```

```
                tmp[2] = k[i][2];

                if (tmp[1] < tmp[0]) swap(tmp[1], tmp[0]);
                if (tmp[2] < tmp[1]) swap(tmp[2], tmp[1]);

                if (i % prime[j] == 0) {
                    break;
                }
            }
        }

        for (int i = 0; i <= RADIO; i++) {
            sg[i][0] = sg[0][i] = i;
        }

        for (int i = 1; i <= RADIO; i++) {
            for (int j = 1; j <= i; j++) {
                sg[i][j] = sg[j][i] = sg[j][i % j];
            }
        }
    }

    int query(int a, int b) const {
        if (a == 0) return b;
        if (b == 0) return a;

        int g = 1;
        for (int i = 0; i < 3; ++i) {
            int ka = k[a][i];
            if (ka == 1) continue;

            int cf;
            if (ka > RADIO) {
                cf = (b % ka == 0) ? ka : 1;
            } else {
                cf = sg[ka][b % ka];
            }
            g *= cf;
            b /= cf;
        }
        return g;
    }
};
```

2.17 向下取整与向上取整

```
template <class T>
T floor_div(const T &a, const T &b) {
    assert(b != 0);
    T q = a / b;
    T r = a % b;
    if (r != 0 && ((r > 0) != (b > 0))) {
        --q;
    }
    return q;
}

template <class T>
T ceil_div(const T &a, const T &b) {
    assert(b != 0);
    T q = a / b;
    T r = a % b;
    if (r != 0 && ((r > 0) == (b > 0))) {
        ++q;
    }
    return q;
}
```

3 字符串

STRINGS

3.1 KMP

> 下标从 0 开始。

```cpp
vector<int> pre_function(const string &t) {
    int m = t.size();
    vector<int> pi(m);
    for (int i = 1; i < m; i++) {
        int j = pi[i - 1];
        while (j > 0 && t[i] != t[j]) {
            j = pi[j - 1];
        }
        if (t[i] == t[j]) {
            j++;
        }
        pi[i] = j;
    }
    return pi;
}

vector<int> KMP(const string &s, const string &t) {
    int n = s.size(), m = t.size();
    vector<int> pi = pre_function(t);
    vector<int> res;
    for (int i = 0, j = 0; i < n; i++) {
        while (j > 0 && s[i] != t[j]) {
            j = pi[j - 1];
        }
        if (s[i] == t[j]) {
            j++;
        }
        if (j == m) {
            res.push_back(i - j + 1);
            j = pi[j - 1];
        }
    }
    return res;
}
```

3.2 字符串哈希（随机底数）

```cpp
constexpr u64 mod = (1ull << 61) - 1;
mt19937_64 rnd(chrono::steady_clock::now().time_since_epoch().count());
uniform_int_distribution<u64> dist(mod / 2, mod - 2);
const u64 base = dist(rnd);

struct StringHash {
    vector<u64> h;
    vector<u64> p;
    StringHash() {}
    StringHash(const string &s) {
        init(s);
    }
    static u64 add(u64 a, u64 b) {
        a += b;
        if (a >= mod) a -= mod;
        return a;
    }
    static u64 mul(u64 a, u64 b) {
        u128 c = u128(a) * b;
        return add(c >> 61, c & mod);
    }
    void init(const string &s) {
        int n = s.size() - 1;
        p.resize(n + 1);
        h.resize(n + 1);
        p[0] = 1;
        for (int i = 1; i <= n; i++) {
            p[i] = mul(p[i - 1], base);
            h[i] = mul(h[i - 1], base);
            h[i] = add(h[i], s[i]);
        }
    }
    u64 get(int l, int r) {
        return add(h[r], mod - mul(h[l - 1], p[r - l + 1]));
    }
};
```

3.3 字符串哈希（随机底数与模数）

```cpp
bool isPrime(int n) {
    if (n <= 1) return false;
    for (int i = 2; i <= n / i; i++) {
        if (n % i == 0) return false;
    }
    return true;
}

int findPrime(int n) {
    while (!isPrime(n)) {
        n++;
    }
    return n;
}

std::mt19937 rng(std::chrono::steady_clock::now().time_since_epoch().count());
const int P = findPrime(rng() % 900'000'000 + 900'000'000),
base = std::uniform_int_distribution<int>(8e8, 9e8)(rng);

struct Hash {
    vector<ll> h;
    vector<ll> p;
    Hash(){}
    Hash(string &s) {
        init(s);
    }
    void init(string &s) {
        int n = s.size();
        s = " " + s;
        p.resize(n + 1);
        h.resize(n + 1);
        p[0] = 1;
        for (int i = 1; i <= n; i++) {
            p[i] = p[i - 1] * base % P;
            h[i] = (h[i - 1] * base + s[i]) % P;
        }
    }
    ll get(int l, int r) {
        return (h[r] - h[l - 1] * p[r - l + 1] % P + P) % P;
    }
};
```

3.4 字典树

```cpp
constexpr int N = 1e6;

int trie[N][26];
int tot = 0;

void clear() {
    for (int i = 0; i <= tot; i++) {
        fill(trie[i], trie[i] + 26, 0);
    }
    tot = 0;
}

void insert(const string &s) {
    int n = s.size();
    int p = 0;
    for (int i = 0; i < n; i++) {
        int &nxt = trie[p][s[i] - 'a'];
        if (nxt == 0) {
            nxt = ++tot;
        }
        p = nxt;
    }
}
```

3.5 01 字典树

> 补充整数最大异或查询变体。

```cpp
constexpr int N = 1e7;
int tot = 0;
int trie[N][2];

// 多测qin
void clear() {
    for (int i = 0; i <= tot; i++) {
        fill(trie[i], trie[i] + 2, 0);
    }
```

```
    tot = 0;
}

void insert(int x) {
    int p = 0;
    for (int i = 30; i >= 0; i--) {
        int &nxt = trie[p][x >> i & 1];
        if (nxt == 0) {
            nxt = ++tot;
        }
        p = nxt;
    }
}

// 查询最大异或对
int queryMax(int x) {
    int res = 0;
    int p = 0;
    for (int i = 30; i >= 0; i--) {
        int d = x >> i & 1;
        int &nxt = trie[p][d ^ 1];
        if (nxt == 0) {
            p = trie[p][d];
        } else {
            p = nxt;
            res += 1LL << i;
        }
    }
    return res;
}
```

3.6 Z 函数

> 下标从 0 开始。

```
vector<int> z_algorithm(const string &s) {
    int n = s.size();
    vector<int> z(n);
    int l = 0, r = 0;
    for (int i = 1; i < n; i++) {
        if (i <= r) {
            z[i] = min(z[i - l], r - i + 1);
        }
        while (i + z[i] < n && s[z[i]] == s[i + z[i]]) {
            l = i, r = i + z[i];
            z[i]++;
        }
    }
    return z;
}
```

3.7 Manacher

> 下标从 0 开始。

```
vector<int> manacher(const string &s) {
    string t = "#";
    for (auto c : s) {
        t.push_back(c);
        t.push_back('#');
    }
    int n = t.size();
    vector<int> r(n);
    for (int i = 0, j = 0; i < n; i++) {
        if (j * 2 - i >= 0 && j + r[j] > i) {
            r[i] = min(r[j * 2 - i], j + r[j] - i);
        }
        while (i - r[i] >= 0 && i + r[i] < n && t[i - r[i]] == t[i + r[i]]) {
            r[i]++;
        }
        if (i + r[i] > j + r[j]) {
            j = i;
        }
    }
    return r;
}
```

4 图与树

GRAPHS & TREES

4.1 Dijkstra

```cpp
constexpr i64 inf = numeric_limits<i64>::max() / 3;
vector<i64> dijkstra(const vector<vector<array<int, 2>>> &adj, int s) {
    int n = int(adj.size()) - 1;
    vector<i64> dis(n + 1, inf);
    dis[s] = 0;
    priority_queue<pair<i64, int>, vector<pair<i64, int>>, greater<>> pq;
    pq.push({0, s});
    while (!pq.empty()) {
        auto [d, u] = pq.top();
        pq.pop();
        if (d != dis[u]) {
            continue;
        }
        for (auto [w, v] : adj[u]) {
            if (dis[u] + w < dis[v]) {
                dis[v] = dis[u] + w;
                pq.push({dis[v], v});
            }
        }
    }
    return dis;
}
```

4.2 Floyd

```cpp
vector dis(n + 1, vector<ll>(n + 1, inf));
for (int i = 1; i <= m; i++) {
    if (era[i]) continue;
    auto [u, v, w] = e[i];
    dis[u][v] = w;
    dis[v][u] = w;
}
for (int i = 1; i <= n; i++) {
    dis[i][i] = 0;
}
for (int k = 1; k <= n; k++) {
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= n; j++) {
            dis[i][j] = min(dis[i][k] + dis[k][j], dis[i][j]);
        }
    }
}
```

4.3 SPFA

```cpp
constexpr i64 inf = 1e18;
vector<i64> spfa(const vector<vector<array<int, 2>>> &adj, int s) {
    int n = adj.size() - 1;
    vector<i64> dis(n + 1, inf);
    vector<int> cnt(n + 1);
    vector<bool> vis(n + 1);
    dis[s] = 0;
    vis[s] = true;
    queue<int> q;
    q.push(s);
    while (!q.empty()) {
        int u = q.front();
        q.pop();
        vis[u] = false;
        for (auto [w, v] : adj[u]) {
            if (dis[v] > dis[u] + w) {
                dis[v] = dis[u] + w;
                cnt[v] = cnt[u] + 1;
                // 判断负环，要写≥点的个数，如果0位置也有点要写≥n+1
                if (cnt[v] >= n) {
                    return {};
                }
                if (!vis[v]) {
                    q.push(v);
                    vis[v] = true;
                }
            }
        }
    }
    return dis;
}
```

4.4 差分约束

> 约束 a - b ≤ c 转化为一条 b → a、边权为 c 的有向边。

```
int n, m;
cin >> n >> m;
vector<vector<array<int, 2>>> adj(n + 1);
for (int i = 0; i < m; i++) {
    int u, v, w;
    cin >> u >> v >> w;
    adj[v].push_back({w, u});
}
for (int i = 1; i ≤ n; i++) {
    adj[0].push_back({0, i});
}
// 此时dis中就是一组解，如果为空，代表无解
auto dis = spfa(adj, 0);
```

4.5 LCA（倍增）

```
int N = __lg(n - 1);
vector<int> dep(n + 1);
vector f(n + 1, vector<int>(N + 1));

auto dfs = [&](auto &&dfs, int u, int p) -> void {
    dep[u] = dep[p] + 1;
    f[u][0] = p;
    for (int i = 1; i ≤ N; i++) {
        f[u][i] = f[f[u][i - 1]][i - 1];
    }
    for (auto v : adj[u]) {
        if (v == p) continue;
        dfs(dfs, v, u);
    }
};
dfs(dfs, s, 0);

auto lca = [&](int x, int y) -> int {
    if (dep[x] < dep[y]) swap(x, y);
    int d = dep[x] - dep[y];
    for (int i = __lg(d); i ≥ 0; i--) {
        if (d >> i & 1) {
            x = f[x][i];
        }
    }

    if (x == y) return x;
    for (int i = N; i ≥ 0; i--) {
        if (f[x][i] ≠ f[y][i]) {
            x = f[x][i];
            y = f[y][i];
        }
    }
    return f[x][0];
};
```

4.6 LCA（DFS 序）

```
vector<int> dfn(n + 1), seg(n + 1), siz(n + 1), par(n + 1), dep(n + 1);
int tot = 0;
auto dfs = [&](auto &&dfs, int u, int p) -> void {
    seg[++tot] = u;
    dfn[u] = tot;
    dep[u] = dep[p] + 1;
    par[u] = p;
    siz[u] = 1;
    for (auto v : adj[u]) {
        if (v == p) {
            continue;
        }
        dfs(dfs, v, u);
        siz[u] += siz[v];
    }
};
dfs(dfs, r, 0);

RMQ<int> rmq(seg, [&](int x, int y) {
    if (dep[x] < dep[y]) {
        return x;
    }
    return y;
});
```

```
auto lca = [&](int x, int y) -> int {
    if (dfn[x] > dfn[y]) {
        swap(x, y);
    }

    if (dfn[x] + siz[x] - 1 >= dfn[y] + siz[y] - 1) {
        return x;
    }

    return par[rmq.query(dfn[x], dfn[y])];
};
```

4.7 树链剖分

```
struct HLD {
    int n;
    int cur;
    vector<vector<int>> adj;
    vector<int> siz, par, hvy, dep, dfn, seq, top, out;
    HLD() {}
    HLD(int N) {
        cur = 0;
        n = N;
        adj.assign(n + 1, {});
        siz.resize(n + 1);
        par.resize(n + 1);
        hvy.resize(n + 1);
        dep.resize(n + 1);
        dfn.resize(n + 1);
        seq.resize(n + 1);
        top.resize(n + 1);
        out.resize(n + 1);
    }
    void work(int root = 1) {
        dfs1(root, 0);
        dfs2(root, 0, root);
        for (int i = 1; i <= n; i++) {
            out[i] = dfn[i] + siz[i] - 1;
        }
    }
    void addEdge(int u, int v) {
        adj[u].push_back(v);
        adj[v].push_back(u);
    }
    void dfs1(int u, int p) {
        par[u] = p;
        siz[u] = 1;
        dep[u] = dep[p] + 1;
        int mx = 0, hc = 0;
        for (auto v : adj[u]) {
            if (v == p) {
                continue;
            }
            dfs1(v, u);
            siz[u] += siz[v];
            if (siz[v] > mx) {
                mx = siz[v];
                hc = v;
            }
        }
        hvy[u] = hc;
    }
    void dfs2(int u, int p, int t) {
        seq[++cur] = u;
        dfn[u] = cur;
        top[u] = t;
        if (hvy[u] != 0) {
            dfs2(hvy[u], u, t);
        }
        for (auto v : adj[u]) {
            if (v == p || v == hvy[u]) {
                continue;
            }
            dfs2(v, u, v);
        }
    }
    bool isAncester(int x, int y) {
        return (dfn[x] <= dfn[y]) && (out[x] >= out[y]);
    }
    int lca(int x, int y) {
        while (top[x] != top[y]) {
            if (dep[top[x]] < dep[top[y]]) {
                swap(x, y);
            }
```

```
            x = par[top[x]];
        }
        return dep[x] <= dep[y] ? x : y;
    }
    int dis(int x, int y) {
        return dep[x] + dep[y] - dep[lca(x, y)] * 2;
    }
    vector<pair<int, int>> getPath(int x, int y) {
        vector<pair<int, int>> res;
        while (top[x] != top[y]) {
            if (dep[top[x]] < dep[top[y]]) {
                swap(x, y);
            }
            res.emplace_back(dfn[top[x]], dfn[x]);
            x = par[top[x]];
        }
        res.emplace_back(min(dfn[x], dfn[y]), max(dfn[x], dfn[y]));
        return res;
    }
    auto tie_arrays() {
        return tie(dfn, siz, dep, par, top, out);
    }
};
```

4.8 克鲁斯卡尔重构树

```
int n, m;
cin >> n >> m;
HLD t(n * 2);
vector<array<int, 3>> e(m);
for (int i = 0; i < m; i++) {
    int u, v, w;
    cin >> u >> v >> w;
    e[i] = {w, u, v};
}
sort(all(e));

int tot = n;
DSU dsu(n * 2);
vector<int> val(n * 2);
for (auto [w, u, v] : e) {
    if (dsu.same(u, v)) {
        continue;
    }
    int par = ++tot;
    u = dsu.find(u);
    v = dsu.find(v);
    t.addEdge(par, u);
    t.addEdge(par, v);
    dsu.merge(par, u);
    dsu.merge(par, v);
    val[par] = w;
}

for (int i = 1; i < n * 2; i++) {
    if (i == dsu.find(i)) {
        t.work(i);
    }
}
```

4.9 树哈希

```
mt19937_64 rnd(chrono::steady_clock::now().time_since_epoch().count());
const u64 mask = rnd();
struct TreeHash {
    int n;
    vector<u64> h, rt;
    vector<vector<int>> adj;
    TreeHash(int N): n(N), adj(N + 1), h(N + 1), rt(N + 1) {}
    static u64 shift(u64 x) {
        x ^= mask;
        x ^= x << 13;
        x ^= x >> 7;
        x ^= x << 17;
        x ^= mask;
        return x;
    }
    void addEdge(int u, int v) {
        adj[u].push_back(v);
        adj[v].push_back(u);
    }
    void dfs1(int u, int p = 0) {
        h[u] = 1;
```

```
        for (auto v : adj[u]) {
            if (v == p) {
                continue;
            }
            dfs1(v, u);
            h[u] += shift(h[v]);
        }
    }
    void dfs2(int u, int p = 0) {
        for (auto v : adj[u]) {
            if (v == p) {
                continue;
            }
            rt[v] = h[v] + shift(rt[u] - shift(h[v]));
            dfs2(v, u);
        }
    }
    void work(int r = 1) {
        dfs1(r);
        rt[r] = h[r];
        dfs2(r);
    }
    auto tie_arrays() {
        return tie(h, rt);
    }
};
```

4.10 树的重心

```
vector<int> siz(n + 1);
vector<int> g;
{
    auto dfs = [&](auto &&dfs, int u, int p = 0) -> void {
        siz[u] = 1;
        int mx = 0;
        for (auto v : adj[u]) {
            if (v == p) {
                continue;
            }
            dfs(dfs, v, u);
            siz[u] += siz[v];
            mx = max(mx, siz[v]);
        }
        mx = max(mx, n - siz[u]);

        if (mx <= n / 2) {
            g.push_back(u);
        }
    };
    dfs(dfs, 1);
}
```

4.11 Hierholzer（无向图）

```
// 无向图
vector<int> Hierholzer(vector<vector<int>> adj) {
    int n = int(adj.size()) - 1;
    int odd = 0;
    vector<int> deg(n + 1);
    for (int u = 1; u <= n; u++) {
        for (auto v : adj[u]) {
            deg[u]++;
            if (u == v) {
                deg[u]++;
            }
        }
    }
    int s = 0;
    for (int i = n; i >= 1; i--) {
        if (deg[i] & 1) {
            odd++;
            s = i;
        }

        if (s == 0 && deg[i] != 0) {
            s = i;
        }
    }

    if (odd != 0 && odd != 2) {
        return {};
    }
```

```
    vector<int> path;
    vector<int> stk;
    stk.push_back(s);

    while (!stk.empty()) {
        int u = stk.back();
        if (adj[u].empty()) {
            path.push_back(u);
            stk.pop_back();
        } else {
            int v = adj[u].back();
            adj[u].pop_back();
            auto it = find(adj[v].begin(), adj[v].end(), u);
            if (it != adj[v].end()) {
                *it = adj[v].back();
                adj[v].pop_back();
            }
            stk.push_back(v);
        }
    }

    for (int u = 1; u <= n; u++) {
        if (!adj[u].empty()) {
            return {};
        }
    }

    reverse(path.begin(), path.end());
    return path;
}
```

4.12 Hierholzer（有向图）

```
// 有向图
// 保证至少有一条边即可，允许自环和重边
vector<int> Hierholzer(vector<vector<int>> adj) {
    int n = int(adj.size()) - 1;
    vector<int> in(n + 1), out(n + 1);
    for (int u = 1; u <= n; u++) {
        for (auto v : adj[u]) {
            in[v]++;
            out[u]++;
        }
    }

    int s = 0;
    int x = 0, y = 0;
    for (int i = 1; i <= n; i++) {
        if (abs(in[i] - out[i]) > 1) {
            return {};
        }

        if (s == 0 && in[i] == out[i] && in[i] != 0) {
            s = i;
        }

        if (out[i] - in[i] == 1) {
            x++;
            s = i;
        } else if (in[i] - out[i] == 1) {
            y++;
        }
    }

    if ((x != 0 || y != 0) && (x != 1 || y != 1)) {
        return {};
    }

    vector<int> stk;
    vector<int> path;
    stk.push_back(s);

    while (!stk.empty()) {
        int u = stk.back();
        if (adj[u].empty()) {
            path.push_back(u);
            stk.pop_back();
        } else {
            int v = adj[u].back();
            stk.push_back(v);
            adj[u].pop_back();
        }
    }
```

```
    for (int u = 1; u <= n; u++) {
        if (!adj[u].empty()) {
            return {};
        }
    }

    reverse(path.begin(), path.end());
    return path;
}
```

4.13 强连通分量

```
struct SCC {
    int n;
    vector<vector<int>> adj;
    vector<int> stk;
    vector<int> dfn, low, bel;
    int cur, cnt;

    SCC() {}
    SCC(int N) {
        init(N);
    }

    void init(int N) {
        n = N;
        adj.assign(n + 1, {});
        dfn.assign(n + 1, -1);
        low.resize(n + 1);
        bel.assign(n + 1, -1);
        stk.clear();
        cur = cnt = 0;
    }

    void addEdge(int u, int v) {
        adj[u].push_back(v);
    }

    void dfs(int x) {
        dfn[x] = low[x] = cur++;
        stk.push_back(x);

        for (auto y : adj[x]) {
            if (dfn[y] == -1) {
                dfs(y);
                low[x] = min(low[x], low[y]);
            } else if (bel[y] == -1) {
                low[x] = min(low[x], dfn[y]);
            }
        }

        if (dfn[x] == low[x]) {
            int y;
            do {
                y = stk.back();
                bel[y] = cnt + 1;
                stk.pop_back();
            } while (y != x);
            cnt++;
        }
    }

    vector<int> work() {
        for (int i = 1; i <= n; i++) {
            if (dfn[i] == -1) {
                dfs(i);
            }
        }
        return bel;
    }
};
```

5 常用工具

UTILITIES

5.1 chmin / chmax

```cpp
template <class T>
bool chmin(T &a, const T &b) {
  if (b < a) {
    a = b;
    return true;
  }
  return false;
}

template <class T>
bool chmax(T &a, const T &b) {
  if (b > a) {
    a = b;
    return true;
  }
  return false;
}
```

5.2 int128 输入输出与运算

```cpp
 std::ostream &operator<<(std::ostream &os, i128 n) {
    if (n == 0) {
        return os << 0;
    }
    std::string s;
    while (n > 0) {
        s += char('0' + n % 10);
        n /= 10;
    }
    std::reverse(s.begin(), s.end());
    return os << s;
}

i128 toi128(const std::string &s) {
    i128 n = 0;
    for (auto c : s) {
        n = n * 10 + (c - '0');
    }
    return n;
}

i128 sqrti128(i128 n) {
    i128 lo = 0, hi = 1E16;
    while (lo < hi) {
        i128 x = (lo + hi + 1) / 2;
        if (x * x <= n) {
            lo = x;
        } else {
            hi = x - 1;
        }
    }
    return lo;
}

i128 gcd(i128 a, i128 b) {
    while (b) {
        a %= b;
        std::swap(a, b);
    }
    return a;
}
```

5.3 编译脚本

```sh
#!/bin/bash

g++ -std=c++20 -O2 -Wall $1.cpp -o $1

./$1 < $1.in > $1.out

cat $1.out
```

5.4 对拍（Linux）

```cpp
#include <bits/stdc++.h>
using namespace std;
int main() {
    // 编译三个文件
    system("g++ -std=c++2a main.cpp -o main");
    system("g++ -std=c++2a main__Good.cpp -o main__Good");
    system("g++ -std=c++2a main__Generator.cpp -o main__Generator");

    int t = 0;
    while (true) {
        cout << "test: " << t++ << '\n';

        system("./main__Generator > test.in");

        system("./main < test.in > bad.txt");
        system("./main__Good < test.in > good.txt");

        if (system("diff bad.txt good.txt")) {
          cout << "wrong answer\n";
          return 0;
        }
    }
}
```

5.5 随机数据生成

```cpp
mt19937 rnd(chrono::steady_clock::now().time_since_epoch().count());

// 生成 [l, r] 范围内的数
int rng(int l, int r) {
    return rnd() % (r - l + 1) + l;
}

// 生成在 [l, r] 范围内的一个区间
pair<int, int> interval(int l = 1, int r = 5) {
    int x = rng(l, r);
    int y = rng(l, r);
    return minmax(x, y);
}

// 生成节点数在 [l, r] 范围内的一棵树
void tree(int l = 1, int r = 5) {
    int n = rng(l, r);
    cout << n << '\n';

    for (int u = 2; u <= n; u++) {
        int v = rng(1, u - 1);
        cout << u << " " << v << '\n';
    }
}

// 生成节点数在 [l, r] 范围内的一个无向连通图
void graph(int l = 1, int r = 5) {
    int n = rng(l, r);
    int m = rng(n - 1, n * (n - 1) / 2);
    cout << n << " " << m << '\n';
    set<pair<int, int>> S;
    vector<pair<int, int>> edges;
    for (int u = 2; u <= n; u++) {
        int v = rng(1, u - 1);
        S.insert({u, v});
        edges.push_back({u, v});
    }

    for (int i = n; i <= m; i++) {
        int u, v;
        do {
            u = rng(1, n);
            v = rng(1, n);
        } while (u == v || S.contains({u, v}));
        S.insert({u, v});
        edges.push_back({u, v});
    }
    for (int i = 0; i < m; i++) {
        auto [u, v] = edges[i];
        cout << u << " " << v << '\n';
    }
}
```